

DSA Counting & Mathematical Patterns

A placement-focused reference for combinations, permutations, factorials, subsets, GCD/LCM, modular arithmetic, binary exponentiation, Catalan numbers, inclusion-exclusion, pigeonhole principle, and basic probability.

Topic	Formula / Idea	Main use
Factorial	$n!$	Arrangements
Permutation	$nPr = n! / (n-r)!$	Order matters
Combination	$nCr = n! / (r!(n-r)!)$	Order does not matter
Catalan	$C_n = C(2n,n) / (n+1)$	Valid parentheses, BSTs, trees
Sum 1..n	$n(n+1)/2$	Counting pairs / ranges
Sum of squares	$n(n+1)(2n+1)/6$	Math / loop optimization
GCD	Euclidean algorithm	Divisibility
LCM	$a/\gcd(a,b) * b$	Common multiples
Power	Binary exponentiation	a^n in $O(\log n)$
$nCr \bmod p$	Factorials + modular inverse	Large modular answers

1. Factorial

Meaning: $n!$ means multiplying every positive integer from 1 to n . For example, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

DSA connection: If n distinct objects can be arranged in every possible order, there are $n!$ arrangements. Factorial also appears inside nPr and nCr .

Important: factorial grows extremely quickly, so use long long only for small n .

C++:

```
long long factorial(int n) {
    long long ans = 1;
    for (int i = 2; i <= n; i++)
        ans *= i;
    return ans;
}
```

Example problem: Example problem: How many ways can 5 different books be arranged on a shelf? Answer: $5! = 120$.

2. Permutation — nPr

Meaning: Choose r items from n items when **order matters**.

Formula: $nPr = n! / (n-r)!$

Instead of calculating two factorials, multiply only r terms. This avoids unnecessary work and is usually the better implementation for DSA.

C++:

```
long long permutation(int n, int r) {
    long long ans = 1;
    for (int i = 0; i < r; i++)
        ans *= (n - i);
    return ans;
}
```

Example problem: Example problem: From 5 students, choose a captain and vice-captain. $A \rightarrow B$ is different from $B \rightarrow A$. Answer = $5P2 = 5 \times 4 = 20$.

3. Combination — nCr

Meaning: Choose r items from n items when **order does not matter**.

Formula: $nCr = n! / (r!(n-r)!)$

Use symmetry: $nCr = nC(n-r)$. Therefore calculate only the smaller of r and $n-r$.

C++:

```
long long combination(int n, int r) {
    if (r > n - r)
        r = n - r;

    long long ans = 1;
    for (int i = 1; i <= r; i++)
        ans = ans * (n - r + i) / i;

    return ans;
}
```

Example problem: Example problem: Choose 2 people from 5 for a team. AB and BA represent the same team. Answer = $5C2 = 10$.

4. 2^n — Subsets

Each of n elements has two independent choices: include it or exclude it. Therefore the number of subsets is 2^n .

This is one of the most important counting ideas in DSA. It appears in subset generation, bitmasking, subsequences, and brute-force search over all choices.

C++:

```
long long countSubsets(int n) {
    return 1LL << n;    // safe only for suitable n
}
```

```
}
```

Example problem: Example problem: For {1,2,3}, the subsets are 8 in total: {}, {1}, {2}, {3}, {1,2}, {1,3}, {2,3}, {1,2,3}.

5. GCD and LCM

GCD: Greatest Common Divisor. Euclid's algorithm uses: $\text{gcd}(a,b) = \text{gcd}(b, a \% b)$.

It runs in $O(\log(\min(a,b)))$.

LCM: Least Common Multiple. For positive a,b: $\text{LCM}(a,b) = (a / \text{GCD}(a,b)) \times b$. Divide first to reduce overflow risk.

C++:

```
long long gcd(long long a, long long b) {
    while (b) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}

long long lcm(long long a, long long b) {
    return (a / gcd(a, b)) * b;
}
```

Example problem: Example: $\text{gcd}(48,18) = 6$. $\text{LCM}(12,18) = (12/6) \times 18 = 36$.

6. Modular Arithmetic

In DSA, answers can become extremely large. Problems often ask for the answer modulo M, commonly 10^9+7 .

Basic rules: $(a+b)\%M = ((a\%M)+(b\%M))\%M$ $(a-b)\%M = ((a\%M)-(b\%M)+M)\%M$ $(a \times b)\%M = ((a\%M) \times (b\%M))\%M$

Division is different: you cannot simply divide after taking modulo. For prime M, use the modular inverse.

C++:

```
const long long MOD = 1000000007;

long long addMod(long long a, long long b) {
    return (a % MOD + b % MOD) % MOD;
}

long long mulMod(long long a, long long b) {
    return (a % MOD) * (b % MOD) % MOD;
}
```

Example problem: Example: If a problem asks for 100000! modulo $1e9+7$, you should compute the value modulo MOD during the calculation rather than constructing the enormous exact factorial.

7. Binary Exponentiation

Naively calculating a^n takes $O(n)$ multiplications. Binary exponentiation reduces this to $O(\log n)$ by repeatedly squaring the base.

If n is even: $a^n = (a^{n/2})^2$. If n is odd: take one extra multiplication by a.

This is extremely useful for large powers and modular exponentiation.

C++:

```
long long power(long long a, long long n) {
    long long ans = 1;

    while (n > 0) {
        if (n & 1)
            ans *= a;

        a *= a;
        n >>= 1;
    }

    return ans;
}

long long modPower(long long a, long long n, long long mod) {
    long long ans = 1;
```

```

a %= mod;

while (n > 0) {
    if (n & 1)
        ans = ans * a % mod;

    a = a * a % mod;
    n >>= 1;
}
return ans;
}

```

Example problem: Example: 2^{10} . Instead of 10 multiplications, binary exponentiation uses repeated squaring: 2^2 , 2^4 , 2^8 , then combines the needed powers. Complexity = $O(\log 10)$.

8. Catalan Numbers

Catalan numbers count many recursively structured objects. Formula: $C_n = C(2n, n) / (n+1)$.

First values: 1, 1, 2, 5, 14, 42, 132...

Common DSA applications: valid parentheses arrangements, number of unique BSTs, ways to triangulate a polygon, and several balanced recursive structures.

The recurrence is: $C_n = \sum_{i=0}^{n-1} C_i \times C_{n-i-1}$.

C++:

```

long long catalan(int n) {
    long long ans = 1;

    for (int i = 0; i < n; i++)
        ans = ans * (2 * n - i) / (i + 1);

    return ans / (n + 1);
}

```

Example problem: Example problem: Number of valid parentheses sequences containing 3 pairs. Answer = $C_3 = 5$.

9. Inclusion-Exclusion

Used when counting two or more groups that overlap.

For two sets A and B: $|A \cup B| = |A| + |B| - |A \cap B|$

Why subtract the intersection? Elements in both sets were counted twice, so remove one copy. For three sets, add pairwise intersections and subtract the triple intersection.

C++:

```

// Two-set inclusion-exclusion
long long unionCount(long long A, long long B, long long intersection) {
    return A + B - intersection;
}

```

Example problem: Example problem: In a class, 30 students know C++ and 20 know Java. If 8 know both, how many know at least one? $30 + 20 - 8 = 42$.

10. Pigeonhole Principle

If more than n objects are placed into n boxes, at least one box contains at least two objects. This is useful for proving that a duplicate or collision must exist.

General form: If N objects are distributed among K boxes, some box contains at least $\lceil N/K \rceil$ objects.

C++:

```

// Minimum guaranteed maximum occupancy:
long long guaranteed(long long N, long long K) {
    return (N + K - 1) / K;
}

```

Example problem: Example problem: Among 13 people, at least two must have birthdays in the same month because there are only 12 months. This is a direct pigeonhole argument.

11. Basic Probability

For equally likely outcomes: **Probability = favorable outcomes / total outcomes.**

In DSA-style counting questions, first count the total possibilities and then count the favorable possibilities using nCr , nPr , or 2^n .

C++:

```
// Probability of choosing a favorable combination
// = favorable / total
// Keep numerator and denominator separately;
// reduce with gcd when an exact fraction is required.
```

Example problem: Example problem: Choose 2 people randomly from 5 people. Total choices = $5C2 = 10$. If 2 specific people form the desired pair, favorable choices = 1. Probability = $1/10$.

12. $nCr \bmod p$

When n is large, the direct factorial formula cannot be used with ordinary integers. For a prime modulus p : $nCr = \text{fact}[n] \times \text{inv}(\text{fact}[r]) \times \text{inv}(\text{fact}[n-r]) \bmod p$. By Fermat's Little Theorem, for nonzero a modulo prime p : $a^{(-1)} = a^{(p-2)} \bmod p$. This method is standard in competitive programming when n is manageable for a precomputed factorial array.

C++:

```
const long long MOD = 1000000007;

long long modPow(long long a, long long e) {
    long long ans = 1;
    while (e) {
        if (e & 1) ans = ans * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return ans;
}

long long nCrMod(long long n, long long r,
                 const vector<long long>& fact) {
    if (r < 0 || r > n) return 0;

    long long res = fact[n];
    res = res * modPow(fact[r], MOD - 2) % MOD;
    res = res * modPow(fact[n-r], MOD - 2) % MOD;
    return res;
}
```

Example problem: Example: Compute $5C2$ modulo $1e9+7$. The normal answer is 10, so the modular answer is also 10.

13. Fast DSA Decision Rules

Ask these questions before coding:

- Does order matter? → permutation.
- Does order not matter? → combination.
- Is every element either selected or not selected? → 2^n / bitmasking.
- Are objects arranged in every order? → factorial.
- Are there overlapping groups? → inclusion-exclusion.
- Must a duplicate/collision exist? → pigeonhole principle.
- Is a power exponent huge? → binary exponentiation.
- Is the answer huge and modulo is given? → modular arithmetic.
- Is the structure balanced/recursive, such as valid parentheses or unique BSTs? → consider Catalan.

C++:

```
// Quick mental mapping
Order matters      -> nPr
Order doesn't matter -> nCr
All arrangements   -> n!
Subsets            -> 2^n
Overlapping sets    -> Inclusion-Exclusion
Duplicate guarantee -> Pigeonhole
Huge exponent      -> Binary exponentiation
Huge modular answer -> Modular arithmetic
```

Valid parentheses $\rightarrow$ Catalan

Example problem: Placement-style mini problems: 1. How many ways to select 3 people from 8? $\rightarrow 8C3$. 2. How many ways to assign president, secretary and treasurer from 8? $\rightarrow 8P3$. 3. How many subsets does an array of 10 distinct values have? $\rightarrow 2^{10}$. 4. Find $\gcd(252, 105)$. $\rightarrow 21$. 5. Compute 3^{100000} efficiently. $\rightarrow$ binary exponentiation. 6. Count valid parentheses with 4 pairs. $\rightarrow C_4 = 14$. 7. Count students knowing at least one of two subjects. $\rightarrow$ inclusion-exclusion. 8. Among 25 people, prove two share a birth month. $\rightarrow$ pigeonhole.

Practice Set

1. Find $7!$ and determine the number of ways to arrange 7 distinct books.
2. From 10 students, choose 4 for a team. Does order matter? Calculate the answer.
3. From 10 students, choose a captain, vice-captain and secretary. Calculate the answer.
4. How many subsets are possible for an array containing 15 distinct elements?
5. Implement gcd and lcm without using `std::gcd`.
6. Calculate 2^{60} using binary exponentiation.
7. Find the 6th Catalan number.
8. A group has 40 students who know Python, 25 who know Java, and 10 who know both. How many know at least one?
9. There are 31 students and 12 months. What is the minimum number of students guaranteed to share a birth month?
10. Compute $100C3$ modulo 1,000,000,007.

Important implementation note: The simple long long formulas are intended for values that fit in long long. For large n , use modular arithmetic, arbitrary precision, or problem-specific combinatorial methods. In competitive programming, always check constraints before choosing a formula.